

TALLER 2: REDES NEURONALES

LADY ALEJANDRA OVALLE JIMENEZ – CÓDIGO: 90478

NICOLAS ANDREY MURILLO MOSQUERA - CÓDIGO: 87640

LAURA CAMILA FIGUEROA PEREZ - CÓDIGO: 140500

INGENIERÍA INDUSTRIAL

DOCENTE: FREDY ALEXANDER ORJUELA LOPEZ

SISTEMAS AVANZADOS DE LA PRODUCCIÓN

UNIVERSIDAD ECCI

INTRODUCCIÓN

En el contexto actual de la ingeniería industrial, caracterizado por la digitalización y el desarrollo de ciudades inteligentes, el análisis avanzado de datos se ha convertido en una herramienta fundamental para la optimización de sistemas productivos y la toma de decisiones estratégicas. En particular, el uso de técnicas de analítica de datos y aprendizaje automático permite modelar comportamientos complejos y extraer conocimiento relevante a partir de grandes volúmenes de información.

El presente informe desarrolla el Taller 2 de Sistemas Avanzados de Producción, el cual se enfoca en la aplicación de métodos analíticos y computacionales para el estudio de la demanda en sistemas de bicicletas compartidas, utilizando el conjunto de datos Capital Bikeshare. Este caso de estudio permite analizar la influencia de variables temporales y factores exógenos, como las condiciones climáticas, sobre la variabilidad de la demanda, planteando un problema real de optimización logística y asignación eficiente de recursos .

El desarrollo del taller se estructura en cinco fases que abarcan el ciclo completo de un proyecto de analítica. En primer lugar, se aborda la regularización Ridge como técnica para controlar la complejidad del modelo y mitigar problemas de sobreajuste. Posteriormente, se estudia el algoritmo de descenso de gradiente como mecanismo de optimización numérica, analizando el impacto de los hiperparámetros en la convergencia del modelo. En la tercera fase, se implementan redes neuronales tipo Perceptrón Multicapa (MLP) y se realiza la optimización de hiperparámetros mediante técnicas como GridSearchCV, evaluando el desempeño a través de métricas como MAE, RMSE y R^2 .

Adicionalmente, se profundiza en la comprensión del funcionamiento interno de las redes neuronales mediante el cálculo manual de la propagación hacia adelante, fortaleciendo el entendimiento conceptual del modelo. Finalmente, se integra el análisis en un contexto industrial a través de un caso de mantenimiento predictivo, donde se evalúa el desempeño de modelos de clasificación bajo condiciones de desbalance de clases, destacando la importancia de métricas como el recall y la precisión en la toma de decisiones operativas.

De esta manera, el presente trabajo no solo aborda el desarrollo técnico de modelos analíticos, sino que también enfatiza su aplicación en escenarios reales de la ingeniería, fortaleciendo competencias en interpretación de resultados, optimización de procesos y toma de decisiones basada en datos, aspectos clave para el desempeño profesional en entornos industriales modernos.

3.1. FASE 1: DESARROLLO ANALITICO: REGULACIÓN RIDGE:

Solución Fase 1: Desarrollo Analítico: Regularización Ridge:

$$J(B) = \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p \beta_j^2$$

① Representación vectorial

$$y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \end{pmatrix} \quad X = \begin{pmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ 1 & x_{31} & x_{32} & \dots & x_{3p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix}$$

② Vector parámetros: B :

$$B = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix} \quad XB = \begin{pmatrix} \beta_0 + \beta_1 x_{11} + \beta_2 x_{12} + \dots + \beta_p x_{1p} \\ \beta_0 + \beta_1 x_{21} + \beta_2 x_{22} + \dots + \beta_p x_{2p} \\ \beta_0 + \beta_1 x_{31} + \beta_2 x_{32} + \dots + \beta_p x_{3p} \\ \vdots \\ \beta_0 + \beta_1 x_{n1} + \beta_2 x_{n2} + \dots + \beta_p x_{np} \end{pmatrix}$$

$$y - XB$$

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad (y - XB)^T (y - XB)$$

$$\lambda \sum_{j=1}^p \beta_j^2 \rightarrow B^T B$$

$$J(B) = (y - XB)^T (y - XB) + \lambda B^T B$$

③ Expansión de la forma cuadrática:

$$(y - x\beta)^T (y - x\beta)$$

→ Propiedad distributiva:

$$(y - x\beta)^T (y - x\beta) = y^T (y - x\beta) - (x\beta)^T (y - x\beta).$$

$$y^T y - y^T x \beta - (x\beta)^T y + (x\beta)^T x \beta$$

→ Simplificar terminos:

$$(x\beta)^T = \beta^T x^T$$

$$(x\beta)^T y = \beta^T x^T y.$$

$$(x\beta)^T x \beta = \beta^T x^T y \beta.$$

$$(y - x\beta)^T (y - x\beta) = y^T y - y^T x \beta - \beta^T x^T y + \beta^T x^T x \beta$$

→ Terminos iguales:

$$\begin{aligned} y^T x \beta &= y^T x \beta \\ \beta^T x^T y &= \beta^T x^T y \\ (y - x\beta)^T (y - x\beta) &= y^T y - 2\beta^T x^T x \beta. \end{aligned}$$

$$x \beta^T \beta$$

$$j(\beta) = y^T y - 2\beta^T x^T y + \beta^T x^T x \beta + \lambda \beta^T \beta$$

$$j(\beta) = y^T x - 2\beta^T x^T + \beta^T (x^T x + \lambda I) \beta.$$

→ Condición de primer orden:

$$j(\beta) = y^T y - z \beta^T x^T + \beta^T (x^T x + \lambda_1) \beta$$

Derivada primer termino:

$$\frac{\partial j(\beta)}{\partial \beta}$$

$$y^T y$$

Derivada segunda termino: $\frac{\partial (y^T y)}{\partial \beta} = 0$

$$-z \beta^T x^T y$$

$$\frac{\partial (\beta^T \alpha)}{\partial \beta} = \alpha$$

$$\alpha = x^T y$$

$$\frac{\partial (-z \beta^T x^T)}{\partial \beta} = -z x^T y$$

Derivada tercer termino: $\beta^T (x^T x + \lambda_1) \beta$

$$\frac{\partial (\beta^T A \beta)}{\partial \beta} = 2 A \beta$$

$$A = x^T x + \lambda_1$$

$$\frac{\partial (\beta^T (x^T x + \lambda_1) \beta)}{\partial \beta} = 2 (x^T x + \lambda_1) \beta$$

① Gradiente completo:

$$\frac{\partial j(B)}{\partial B} = 0 - 2x^T y + 2(x^T x + \lambda_1)B$$

$$\frac{\partial j(B)}{\partial (B)} = -2x^T y + 2(x^T x + \lambda_1)B$$

② Gradiente del vector nulo:

$$\frac{\partial j(B)}{\partial B} = 0.$$

$$-2x^T y + 2(x^T x + \lambda_1)B = 0$$

$$-x^T y + (x^T x + \lambda_1)B = 0$$

$$(x^T x + \lambda_1)B = x^T y.$$

→ Resultado condición primer orden:

$$(x^T x + \lambda_1)B = x^T y.$$

→ Solución cerrada:

$$(x^T x + \lambda_1)B = x^T y.$$

→ Ecuación obtenida:

$$(x^T x + \lambda_1)B = x^T y$$

$$x^T x + \lambda_1$$

→ Multiplicador de inversa de la matriz $(x^T x + \lambda I)$.

$$(x^T x + \lambda I)^{-1} (x^T x + \lambda I) B = (x^T x + \lambda I)^{-1} x^T y.$$

→ Simplificar el lado izquierdo:

$$(x^T x + \lambda I)^{-1} (x^T x + \lambda I) = 1$$

$$1 B = (x^T x + \lambda I)^{-1} x^T y.$$

→ Estimador Ridge:

$$\text{Bridge} \Rightarrow (x^T x + \lambda I)^{-1} x^T y.$$

PREGUNTAS FASE 1:

-SOBRE LA INVERTIBILIDAD:

La suma del término de regularización $\lambda I / \lambda I$ garantiza la invertibilidad de la matriz $(X^T X + \lambda I)$ $(X^T X + \lambda I)$ $(X^T X + \lambda I)$ debido a su efecto sobre los valores propios. Cuando la matriz $X^T X$ es singular, presenta al menos un valor propio igual a cero, lo que impide calcular su inversa.

Al añadir $\lambda I / \lambda I$ con $\lambda > 0$ / $\lambda > 0$, todos los valores propios de $X^T X$ se desplazan en una magnitud λ / λ , es decir, cada valor propio μ_i pasa a ser $\mu_i + \lambda$. Como resultado, todos los valores propios se vuelven estrictamente positivos, lo que convierte a la matriz en definida positiva y, por tanto, invertible.

Esto no solo permite obtener una solución única, sino que también mejora la estabilidad numérica del modelo, especialmente en presencia de multicolinealidad entre variables.

-EFECTO DE PARAMETRO:

El parámetro λ / lambda λ controla el grado de regularización del modelo, afectando directamente el equilibrio entre sesgo y varianza:

-Cuando $\lambda \rightarrow 0$ / lambda / to $0\lambda \rightarrow 0$:

El término de penalización desaparece y el modelo Ridge se aproxima a una regresión por mínimos cuadrados ordinarios (OLS). En este caso, los coeficientes no están restringidos, lo que genera:

- *Bajo sesgo.

- *Alta varianza.

- *Alto riesgo de sobreajuste.

-Cuando $\lambda \rightarrow \infty$ / lambda / to / infty $\lambda \rightarrow \infty$:

La penalización domina completamente la función de costo, forzando a los coeficientes a acercarse a cero. Esto produce:

- *Alto sesgo.

- *Baja varianza.

- *Modelo excesivamente simplificado de subajuste.

En consecuencia, λ / lambda λ actúa como un parámetro de control de la complejidad del modelo, permitiendo encontrar un equilibrio adecuado entre ajuste a los datos y capacidad de generalización. Una correcta selección de este parámetro es clave para obtener modelos robustos en problemas reales de ingeniería.

3.2. FASE 2: OPTIMIZACIÓN NUMÉRICA Y DESCENSO DE GRADIENTE:

Ubique el último bloque de código donde se configura la simulación:

```
# --- CONFIGURACIÓN DE LA SIMULACIÓN ---  
start_point = [0.0, 0.1]    # Punto de inicio  
learning_rate = 0.0001     # Tasa de aprendizaje (eta)  
num_iterations = 1000      # Número de pasos
```

ILUSTRACIÓN 1:

- Descenso del Gradiente en 3D tomada de código semana 6 - Colab.

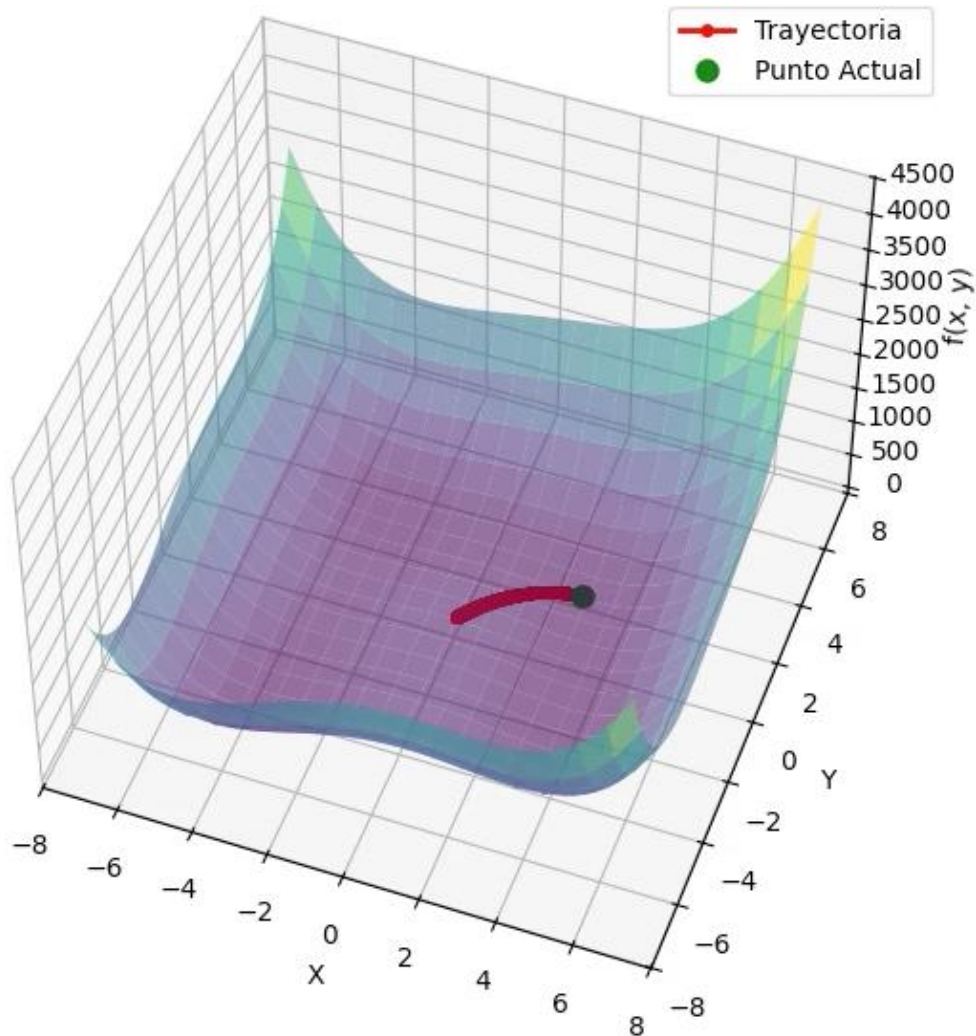
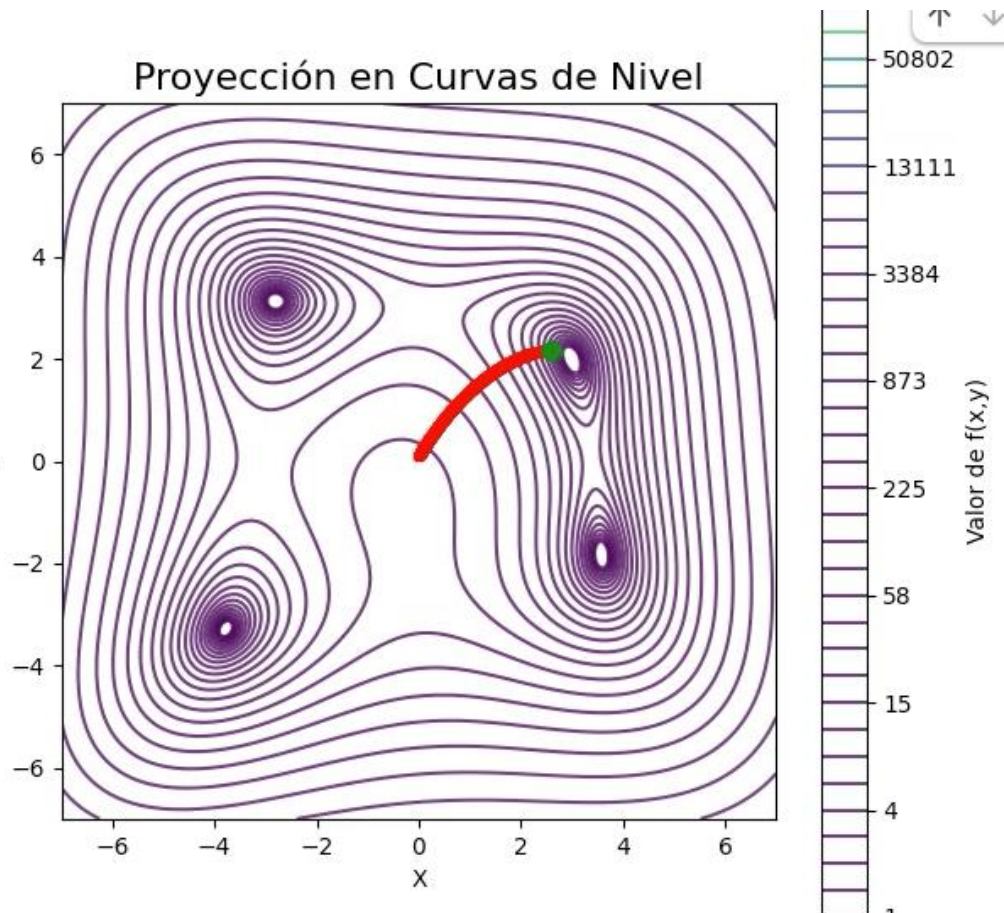


ILUSTRACIÓN 2:

- Proyección en curvas de nivel tomada de código semana 6 - Colab.



TAREAS DE EXPERIMENTACIÓN

1. Impacto de la Tasa de Aprendizaje (η):

- Mantenga el `start_point` y las `num_iterations` constantes.
- Incremente el `learning_rate` a un valor significativamente más alto (ej. 0.5 o 1.0). ¿Qué sucede con la trayectoria? ¿Logra llegar al centro de las curvas de nivel o comienza a oscilar/divergir?

ILUSTRACIÓN 3:

- Descenso del Gradiente en 3D tomada de código semana 6 - Colab.

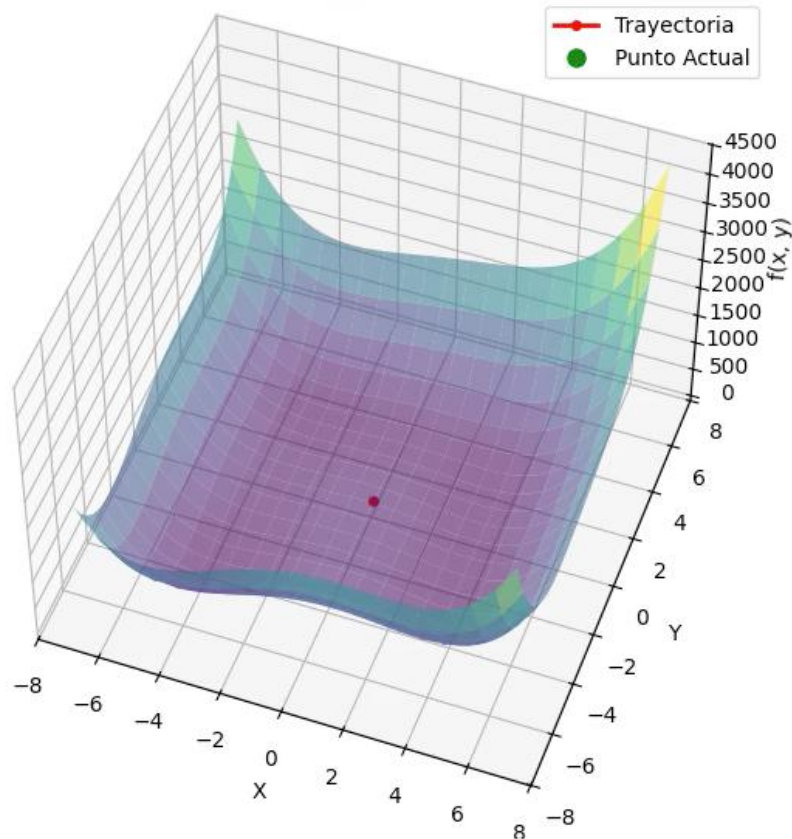


ILUSTRACIÓN 4:

- Proyección en curvas de nivel tomada de código semana 6 - Colab.

RESPUESTA A LAS PREGUNTAS:

Al incrementar la tasa de aprendizaje a un valor elevado como $\eta = 0.6$, manteniendo constante el punto inicial y el número de iteraciones, se evidencia que el algoritmo de descenso de gradiente no logra converger adecuadamente hacia el mínimo de la función. Este comportamiento se explica porque una tasa de aprendizaje alta produce actualizaciones demasiado grandes en cada iteración. En lugar de acercarse progresivamente al mínimo, el algoritmo sobrepasa el punto óptimo en cada paso, generando un fenómeno

conocido como *overshooting*. Como consecuencia, la trayectoria no se estabiliza, sino que comienza a oscilar alrededor del mínimo sin alcanzarlo.

En las gráficas de curvas de nivel, este efecto se visualiza claramente como un patrón en forma de zig - zag, donde los puntos cruzan repetidamente el valle de la función sin converger hacia el centro. Esto indica una falta de estabilidad en el proceso de optimización. En sí, con un valor de $\eta = 0.6$, el algoritmo presenta problemas de convergencia debido a una tasa de aprendizaje excesivamente alta, lo que impide alcanzar el mínimo y compromete la eficiencia del proceso de optimización.

- Reduzca el `learning_rate` a un valor extremadamente pequeño (ej. 10^{-7}). Observe el resultado tras 1000 iteraciones. ¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

ILUSTRACIÓN 5:

- Descenso del Gradiente en 3D tomada de código semana 6 - Colab.

Descenso del Gradiente en 3D

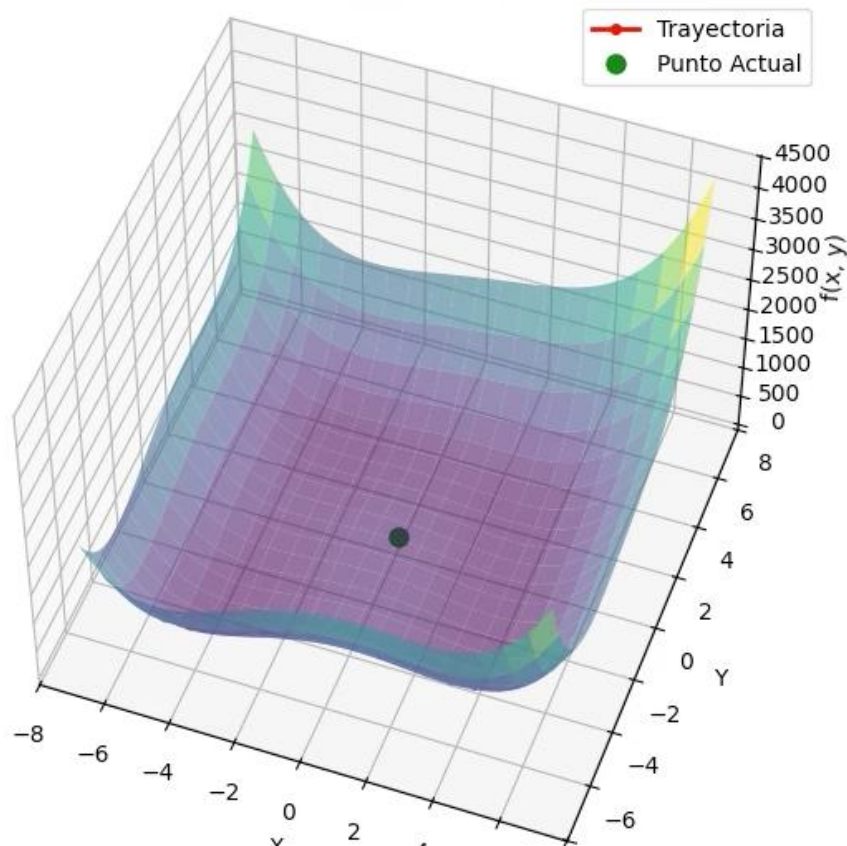
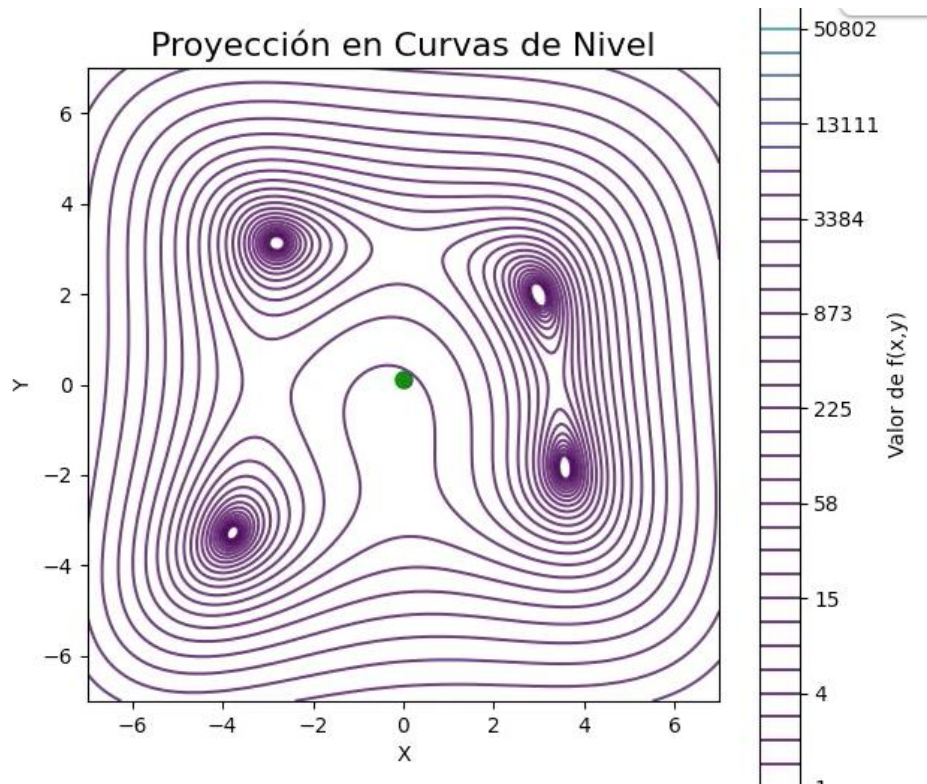


ILUSTRACIÓN 5:

- Proyección en curvas de nivel tomada de código semana 6 - Colab.



RESPUESTA A LA PREGUNTA:

- Al reducir la tasa de aprendizaje a un valor extremadamente pequeño como $\eta = 10^{-7}$, se observa que el algoritmo de descenso de gradiente presenta una convergencia excesivamente lenta hacia el mínimo de la función. Tras 1000 iteraciones, la trayectoria apenas se ha desplazado desde el punto inicial, lo que evidencia que este número de iteraciones no es suficiente para alcanzar el óptimo. Este comportamiento se debe a que el tamaño del paso en cada actualización ($\eta \cdot \nabla f$) es demasiado pequeño, generando cambios prácticamente insignificantes en los valores de las variables en cada iteración.

En las gráficas de superficie y curvas de nivel, este fenómeno se refleja en una trayectoria muy corta y concentrada cerca del punto de partida, sin lograr aproximarse al centro del valle donde se encuentra el mínimo. Aunque el algoritmo es estable y no presenta oscilaciones, una tasa de aprendizaje tan baja compromete seriamente la eficiencia computacional, ya que requiere un número excesivamente alto de iteraciones para converger, lo que lo hace poco práctico en aplicaciones reales.

2. Dependencia del Punto Inicial:

- Modifique el `start_point` a coordenadas alejadas del origen, por ejemplo $[6.0, -6.0]$ o $[-5.0, 5.0]$.
- ¿El algoritmo llega siempre al mismo punto final? Explique si la topografía de la función $f(X, Y)$ sugiere la presencia de múltiples valles o mínimos locales.

ILUSTRACIÓN 7:

- Descenso del Gradiente en 3D tomada de código semana 6 - Colab.

Descenso del Gradiente en 3D

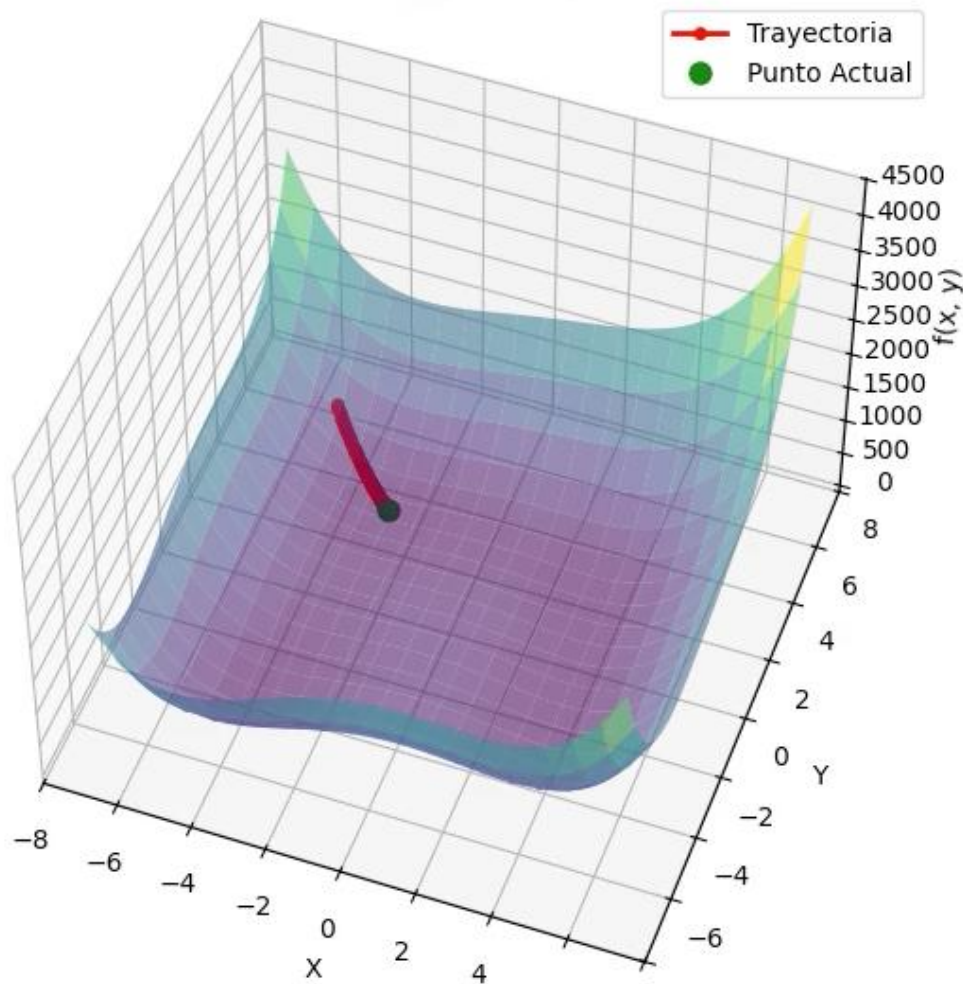
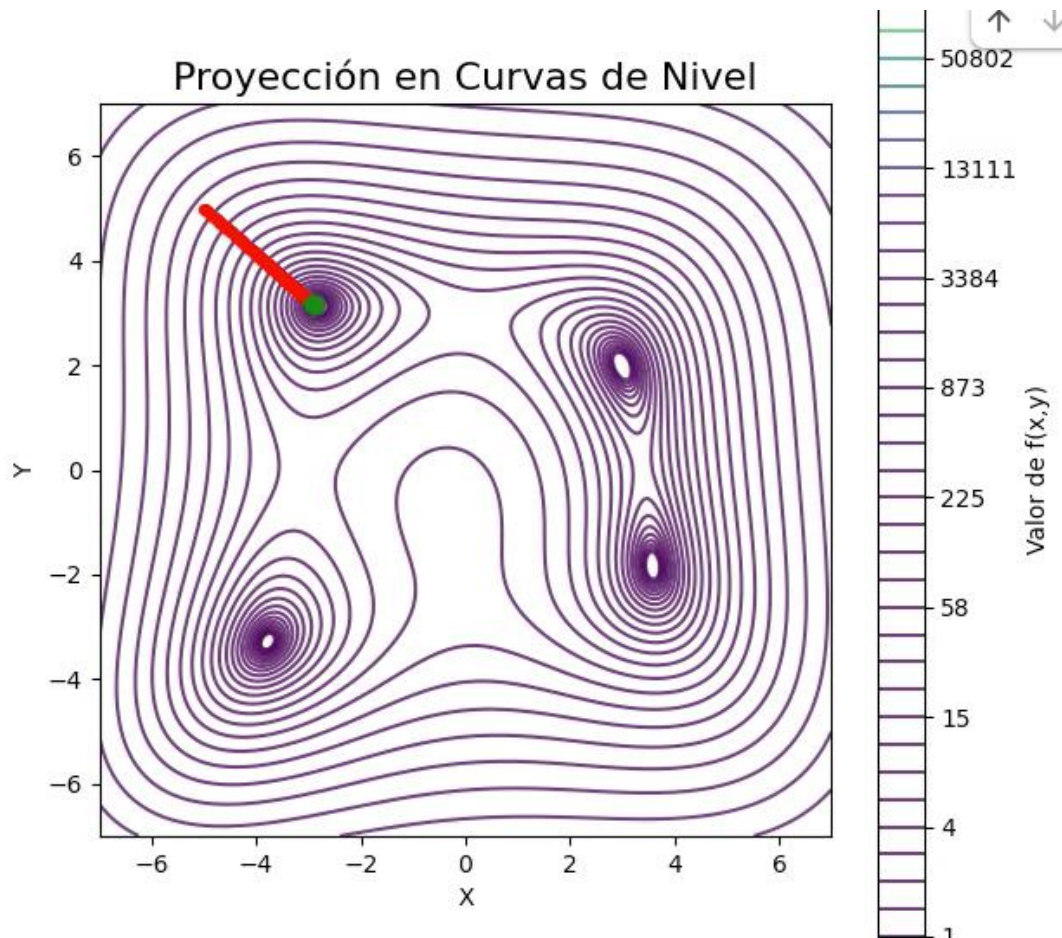


ILUSTRACIÓN 8:

- Proyección en curvas de nivel tomada de código semana 6 - Colab.



RESPUESTA A LAS PREGUNTAS:

-Al modificar el punto inicial a coordenadas alejadas del origen, como $[-5.0, 5.0]$, se observa que el algoritmo de descenso de gradiente logra converger al mismo punto final, independientemente de la posición inicial. Este comportamiento indica que todas las trayectorias generadas por el algoritmo tienden hacia un único mínimo. En las gráficas de superficie y curvas de nivel, esto se evidencia en que, sin importar el punto de partida, los recorridos descienden de manera continua hasta el centro del valle.

Lo anterior sugiere que la función $f(X,Y)$ presenta una topografía convexa, caracterizada por la existencia de un único mínimo global y la ausencia de mínimos locales. Esta propiedad garantiza que el algoritmo no quede atrapado en soluciones subóptimas, lo que asegura la convergencia al óptimo global. En este caso el punto inicial no influye en el

resultado final del algoritmo, sino únicamente en la trayectoria y el número de iteraciones necesarias para alcanzar el mínimo.

Preguntas Fase 2

A partir de sus observaciones en la simulación, responda de manera crítica:

- **El fenómeno de “Overshooting”:** Explique físicamente qué ocurre cuando el paso ($\eta \cdot \nabla f$) es más grande que la distancia al mínimo. ¿Cómo se visualiza esto en el gráfico de curvas de nivel?
- **Eficiencia Computacional:** Si el algoritmo se detiene en la iteración 1000 pero aún se encuentra lejos del mínimo, ¿qué estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.
- **Generalización:** En un problema de ingeniería con cientos de variables (como la optimización de una cadena de suministro), ¿por qué es peligroso confiar en un único punto de inicio aleatorio?

1). El fenómeno de *overshooting* ocurre cuando el tamaño del paso ($\eta \cdot \nabla f$) es mayor que la distancia real al mínimo de la función. Desde una perspectiva física, esto implica que el algoritmo “sobrepasa” el punto óptimo en cada iteración, en lugar de aproximarse de manera progresiva y controlada. Como consecuencia, el descenso de gradiente pierde estabilidad y no logra converger; ya que, en lugar de reducir gradualmente el error, genera oscilaciones alrededor del mínimo. Este comportamiento impide que el algoritmo se establezca en el punto óptimo.

En el gráfico de curvas de nivel, este fenómeno se representa como una trayectoria en forma de zig-zag que atraviesa repetidamente el valle de la función. En lugar de acercarse al centro, los puntos saltan de un lado a otro del mínimo, evidenciando una dinámica inestable típica de una tasa de aprendizaje excesivamente alta.

2). Si el algoritmo alcanza la iteración 1000 y aún se encuentra lejos del mínimo, la estrategia más adecuada es ajustar la tasa de aprendizaje η , en lugar de incrementar únicamente el número de iteraciones. Esto se debe a que una tasa de aprendizaje demasiado baja genera pasos extremadamente pequeños, lo que ralentiza significativamente la convergencia. En este escenario, aumentar el número de iteraciones solo incrementa el costo computacional sin garantizar una mejora sustancial en los resultados.

Por el contrario, ajustar correctamente η permite encontrar un equilibrio entre velocidad y estabilidad en el proceso de optimización, logrando avances más significativos en cada iteración. En consecuencia, una adecuada calibración de la tasa de aprendizaje es clave para mejorar la eficiencia computacional y reducir el tiempo necesario para alcanzar el mínimo.

3). En problemas de ingeniería con alta dimensionalidad, como la optimización de una cadena de suministro, resulta riesgoso depender de un único punto de inicio aleatorio, ya que la función objetivo puede presentar una topografía compleja con múltiples mínimos locales. En este contexto, el algoritmo de descenso de gradiente puede converger hacia un mínimo local que no representa la mejor solución global, lo que conduce a resultados subóptimos y decisiones operativas ineficientes.

Además, la sensibilidad al punto inicial puede afectar la reproducibilidad y la robustez del modelo. Por esta razón, es recomendable emplear múltiples puntos de inicio, técnicas de inicialización controlada o algoritmos más avanzados (como métodos estocásticos o con reinicios), con el fin de aumentar la probabilidad de encontrar el mínimo global y mejorar la calidad de la solución.

3.3. FASE 3: REDES NEURONALES (MLP) Y AJUSTE DE HIPERPARAMETROS:

Parte A: Evaluación de Métricas de Regresión

Ejecute el modelo `MLPRegressor` inicial con la configuración base (`hidden_layer_sizes=(16, 8, 4)`). Observe los resultados de las métricas **MAE**, **RMSE** y R^2 tanto para el conjunto de entrenamiento como para el de prueba (test).

ILUSTRACIÓN 9:

- `MLPRegressor` en dispersión tomado de GitHub: Semana 7 - Redes Neuronales y `GridSearch`. - Colab.

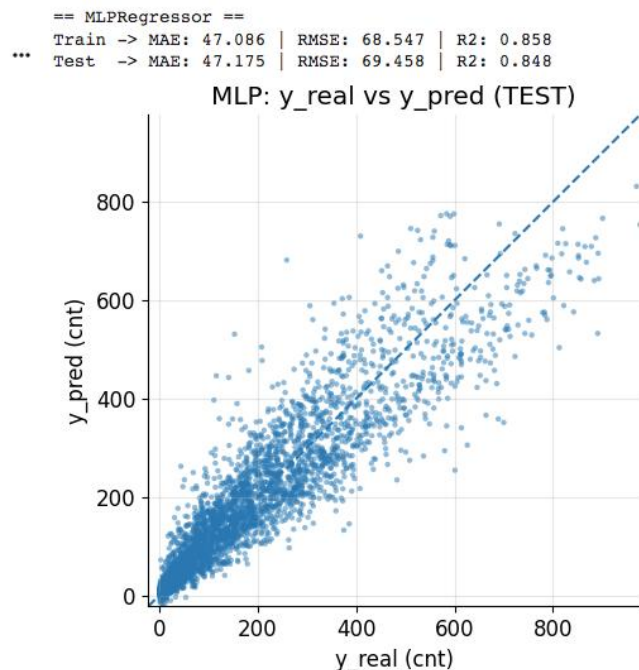


ILUSTRACIÓN 10:

- MLPRegressor en histograma tomado de GitHub: Semana 7 - Redes Neuronales y GridSearch - Colab.

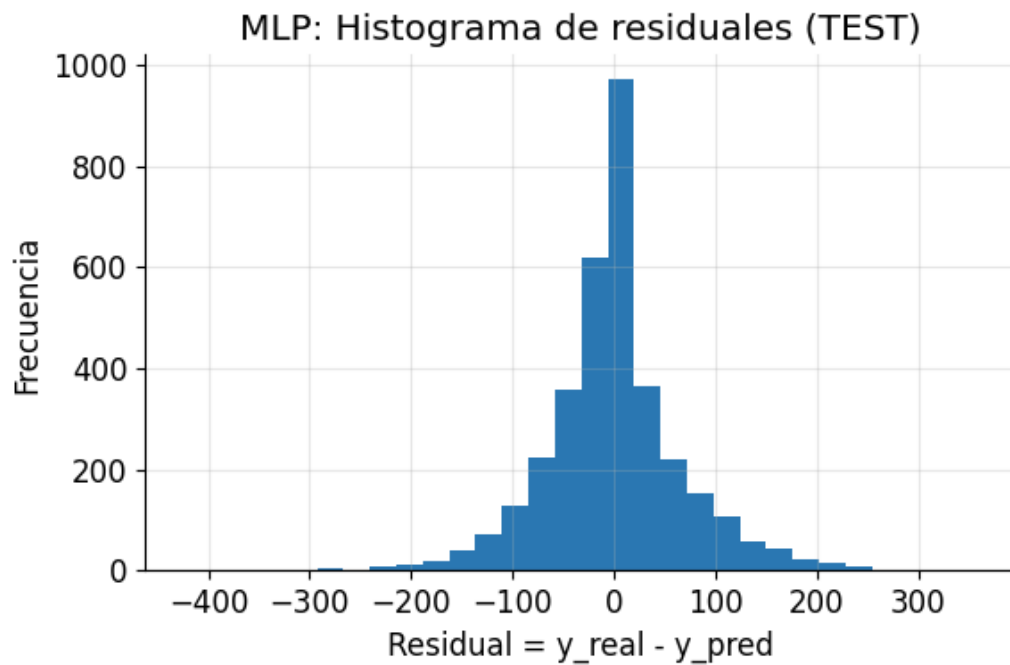
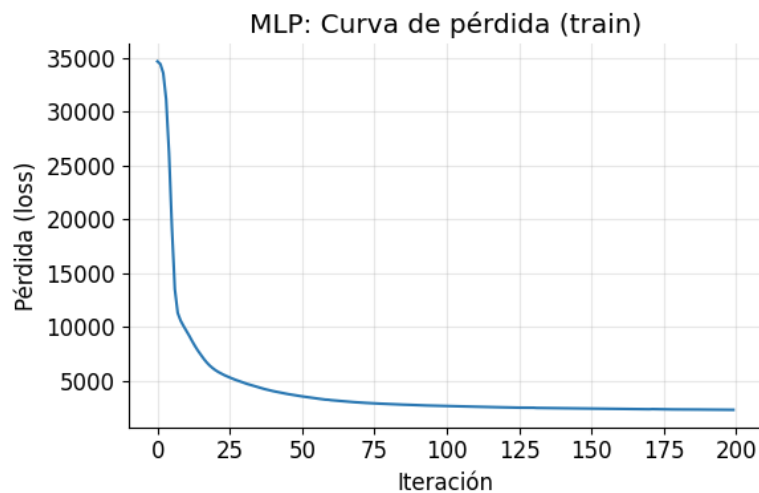


ILUSTRACIÓN 11:

- MLPRegressor en diagrama de curva o tendencia tomado de GitHub: Semana 7 - Redes Neuronales y GridSearch - Colab.



Preguntas Fase 3A:

- **Diferenciación de Métricas:** Explique la diferencia técnica entre el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE). ¿Cuál de las dos penaliza con mayor severidad los errores grandes (*outliers*) en las ventas y por qué?
- **Diagnóstico de Desempeño:** Si el R^2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, ¿qué fenómeno está ocurriendo? Proponga una solución basada en lo aprendido en la Fase 1 del taller.
- **Interpretación de Residuos:** Al observar la gráfica de y_{real} vs y_{pred} , ¿se identifica algún patrón de error sistemático (ej. subestimación en ventas altas)?

RESPUESTA A LAS PREGUNTAS:

1). El Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE) son métricas utilizadas para evaluar el desempeño de modelos de regresión, pero difieren en la forma en que penalizan los errores. El **MAE** calcula el promedio de los errores absolutos entre los valores reales y predichos, es decir, mide la magnitud promedio del error sin considerar su dirección. Es una métrica lineal, por lo que todos los errores tienen el mismo peso. Por otro lado, el **RMSE** eleva al cuadrado los errores antes de promediarlos y posteriormente aplica la raíz cuadrada. Este proceso hace que los errores grandes tengan un impacto mucho mayor en el resultado final.

El **RMSE penaliza con mayor severidad los errores grandes (outliers)**, debido al efecto del término cuadrático. Esto lo convierte en una métrica más sensible a desviaciones significativas, mientras que el MAE es más robusto frente a valores atípicos.

2). Si el coeficiente de determinación R^2 es significativamente mayor en el conjunto de entrenamiento que en el de prueba, se está presentando un fenómeno de **sobreajuste (overfitting)**. Esto implica que el modelo ha aprendido en exceso los patrones específicos del conjunto de entrenamiento, incluyendo ruido, lo que reduce su capacidad de generalización a nuevos datos.

Una solución basada en lo aprendido en la Fase 1 es aplicar **regularización**, específicamente mediante el uso de un parámetro de penalización (como λ en Ridge o α en modelos neuronales). Esta técnica reduce la magnitud de los coeficientes, evitando modelos excesivamente complejos y mejorando la capacidad de generalización. Adicionalmente, se pueden emplear estrategias como validación cruzada, reducción de complejidad del modelo o ajuste de hiperparámetros.

3). Al analizar la gráfica de valores reales (y_{real}) frente a valores predichos (y_{pred}), es posible identificar patrones de error sistemático que indican deficiencias en el modelo. Un comportamiento común es la **subestimación en valores altos de ventas**, donde el modelo tiende a predecir valores menores que los reales. Esto sugiere que el modelo no está capturando adecuadamente la relación en rangos extremos, posiblemente debido a una capacidad limitada o a una regularización excesiva.

Idealmente, los residuos deberían distribuirse de manera aleatoria alrededor de cero, sin patrones visibles. La presencia de tendencias, curvaturas o agrupaciones indica que el modelo no está representando correctamente la estructura de los datos. La identificación de estos patrones permite diagnosticar problemas como falta de complejidad, sesgo en el modelo o necesidad de incluir nuevas variables o transformar las existentes.

3.3.1. Parte B: Optimización con GridSearchCV

La eficacia de una red neuronal depende críticamente de sus hiperparámetros. A continuación, se presenta una guía de referencia completa para orientar su proceso de optimización:

Guía de Referencia: Hiperparámetros del MLPRegressor

Tarea de Experimentación:

Utilizando la Tabla 2 como guía, modifique la parrilla de parámetros (`param_grid`) en el notebook y ejecute la búsqueda sistemática (*GridSearch*). Se sugiere variar estratégicamente los siguientes elementos:

1. **Arquitectura (`hidden_layer_sizes`):** Compare una red “profunda” (más capas, ej. (32,16,8,4)) contra una “ancha” (una sola capa con muchas neuronas, ej. (64,)).
2. **Regularización (`alpha`):** Conecte esto con la Fase 1. Pruebe valores como [1e-5, 1e-2, 1.0]. ¿Cómo afecta un α muy elevado a la capacidad de aprendizaje?
3. **Tasa de Aprendizaje (`learning_rate_init`):** Conecte esto con la Fase 2. Analice cómo influye en el número de iteraciones necesarias para que el *Early Stopping* se active.

Nota sobre complejidad combinatoria: La grilla base tiene $4 \times 3 \times 3 = 36$ combinaciones $\times 3$ *folds* = 108 ajustes. Si añade `activation` y `batch_size`, supera fácilmente las 500 combinaciones.

-Ajustes realizados en el código:

```
Ejecutando GridSearchCV (esto puede tardar un poco)...
Fitting 3 folds for each of 36 candidates, totalling 108 fits
as celdas de código 1lds for each of 36 candidates, totalling 108 fits
GridSearchCV tomó: 1223.29 s

''' == Resultado de la búsqueda (CV) ==
Mejores hiperparámetros: {'alpha': 1e-05, 'hidden_layer_sizes': (32, 16, 8), 'learning_rate_init': 0.001}
RMSE CV (aprox): 72.82871702592134

== MLP (MEJOR MODELO) ==
Train -> MAE: 44.891 | RMSE: 65.143 | R2: 0.872
Test -> MAE: 47.316 | RMSE: 68.616 | R2: 0.851
```

ILUSTRACIÓN 12:

- Búsqueda De Hiperparámetros Y Evaluación Del Mejor Mlp tomado de GitHub: Semana 7 - Redes Neuronales y GridSearch.

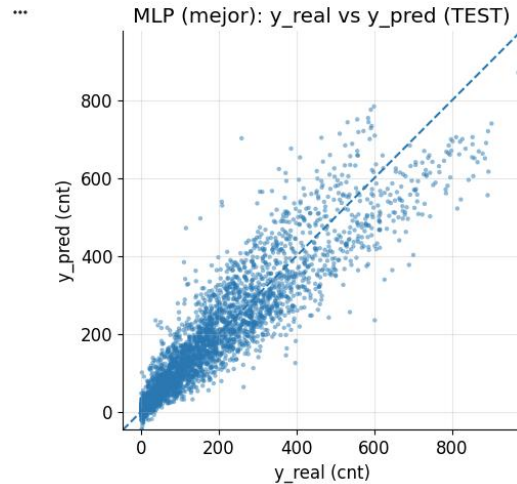


ILUSTRACIÓN 13:

- MLPRegressor en histograma tomado de GitHub: Semana 7 – Redes Neuronales y GridSearch.

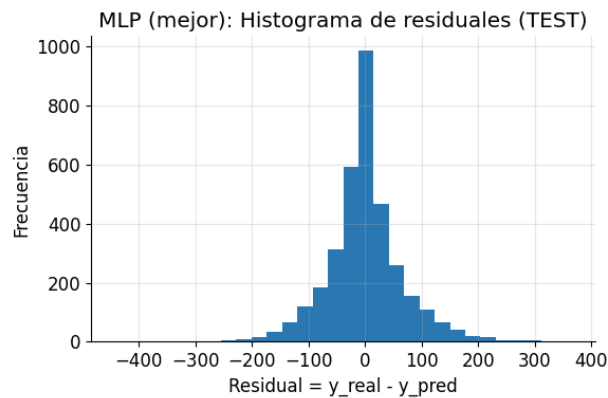
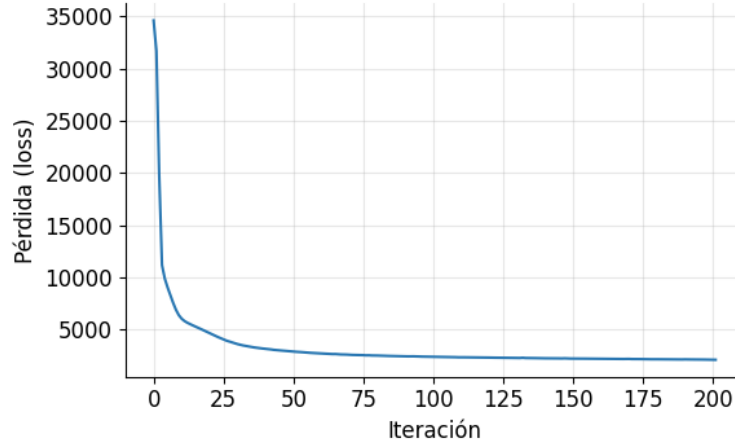


ILUSTRACIÓN 14:

- MLPRegressor en diagrama de curva de perdida tomado de GitHub: Semana 7 - Redes Neuronales y GridSearch.

$$\text{Residual} = y_{\text{real}} - y_{\text{pred}}$$

MLP (mejor): Curva de pérdida (train)



Categoría	Parámetro	Descripción y Valores Sugeridos
Arquitectura	<code>hidden_layer_sizes</code>	Define profundidad y ancho de la red. Valores: [(16,8), (32,16), (64,32), (32,16,8), (64,32,16), (128,64,32)]. Más neuronas aumentan la capacidad pero también el riesgo de sobreajuste.
	<code>activation</code>	Función de activación por capa. <code>'relu'</code> (estándar y robusto), <code>'tanh'</code> (útil con datos centrados), <code>'logistic'</code> , <code>'identity'</code> (regresión lineal pura).
Optimizador	<code>solver</code>	Algoritmo de entrenamiento. <code>'adam'</code> (rápido y adaptativo), <code>'sgd'</code> (permite <i>momentum</i>), <code>'lbfgs'</code> (exacto pero lento; recomendado para datasets pequeños).
	<code>learning_rate_init</code>	Tasa de aprendizaje inicial. Valores: [1e-2, 5e-3, 1e-3, 5e-4, 1e-4]. Una tasa muy alta genera oscilaciones; una muy baja ralentiza la convergencia.
	<code>learning_rate</code>	Esquema de ajuste de LR (<code>sgd</code> únicamente). <code>'constant'</code> , <code>'invscaling'</code> o <code>'adaptive'</code> (reduce LR cuando el loss se estanca).
	<code>momentum</code>	Acumulación de gradiente en pasos previos (<code>sgd</code>). Valores: [0.0, 0.5, 0.9, 0.95]. El valor clásico en deep learning es 0.9.
	<code>batch_size</code>	Tamaño de mini-lote. Valores: [<code>'auto'</code> , 32, 64, 128, 256]. Lotes pequeños introducen más ruido pero suelen mejorar la generalización.
Regularización	<code>alpha</code>	Penalización L_2 sobre los pesos. Valores: [1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1.0]. Controla la complejidad del modelo (análogo a Ridge).
	<code>nesterovs_momentum</code>	Variante de momentum que anticipa el gradiente (<code>sgd</code>). Valores: [True, False]. Generalmente mejora la convergencia.
Early Stopping	<code>early_stopping</code>	Activa la parada anticipada. True/False. Detiene el entrenamiento si el error de validación deja de disminuir, previniendo sobreajuste.
	<code>validation_fraction</code>	Fracción de train usada internamente para monitorear la parada. Valores: [0.05, 0.10, 0.15, 0.20, 0.25].
	<code>n_iter_no_change</code>	"Paciencia": épocas consecutivas sin mejora antes de parar. Valores: [5, 10, 20, 30, 50].
	<code>tol</code>	Mejora mínima para no contabilizar una época como "sin cambio". Valores: [1e-5, 1e-4, 1e-3].
Inicialización	<code>random_state</code>	Semilla de reproducibilidad. Valores: [0, 42, 123]. Usar None introduce variabilidad y permite detectar inestabilidad.
	<code>max_iter</code>	Número máximo de épocas. Valores: [300, 500, 1000, 2000]. Con <code>early_stopping=True</code> raramente se alcanza el tope.

Cuadro 2: Catálogo de hiperparámetros del `MLPRegressor` de scikit-learn para tareas de regresión.

En ese caso, considere `RandomizedSearchCV` con `n_iter=50`.

Preguntas Fase 3B:

- **Consistencia de Parámetros:** Compare el valor óptimo de α encontrado por el GridSearch con lo que analizó en la sección de Regresión Ridge. ¿Existe una relación entre la complejidad de la red y la necesidad de regularización?
- **Early Stopping:** Explique cómo este mecanismo ayuda a optimizar el tiempo de cómputo y a prevenir el sobreajuste basado en sus resultados.
- **Conclusión de Negocio:** Basándose en el mejor modelo encontrado, ¿cuál es el margen de error promedio que la empresa debería esperar al predecir sus ventas? ¿Es el modelo lo suficientemente robusto para decisiones presupuestales? Justifique.

RESPUESTA A LAS PREGUNTAS:

1). El valor óptimo de α obtenido mediante `GridSearchCV` guarda una relación directa con lo analizado en la Regresión Ridge, ya que ambos representan un término de **regularización L2** que penaliza la magnitud de los coeficientes del modelo. En general, se observa que a medida que aumenta la **complejidad de la red neuronal** (mayor número de capas o neuronas), también se incrementa la necesidad de regularización, lo que se traduce en valores de α más altos. Esto se debe a que modelos más complejos tienen mayor capacidad de ajuste, pero también mayor riesgo de sobreajuste.

Por el contrario, redes más simples suelen requerir valores de α más bajos, ya que su capacidad de aprendizaje es limitada y una penalización excesiva podría llevar al subajuste. Existe una relación directa entre la complejidad del modelo y la necesidad de regularización: a mayor complejidad, mayor debe ser el control mediante α , lo que confirma la coherencia entre los resultados del `GridSearch` y los fundamentos teóricos de Ridge.

2). El mecanismo de *Early Stopping* consiste en detener el entrenamiento del modelo cuando el error en el conjunto de validación deja de mejorar durante un número determinado de iteraciones consecutivas. Este método contribuye a **optimizar el tiempo de cómputo**, ya que evita ejecutar iteraciones innecesarias una vez que el modelo ha alcanzado su mejor desempeño. En lugar de agotar el número máximo de épocas, el entrenamiento se detiene automáticamente cuando ya no se obtienen mejoras significativas.

Adicionalmente, el *Early Stopping* actúa como una técnica de **regularización implícita**, ya que previene el sobreajuste. Cuando un modelo continúa entrenándose más allá del punto óptimo, tiende a ajustarse al ruido del conjunto de entrenamiento, deteriorando su capacidad de generalización. Al detener el proceso en el momento adecuado, se conserva un equilibrio

entre ajuste y generalización. En consecuencia, este mecanismo mejora tanto la eficiencia computacional como la robustez del modelo.

3). Con base en el mejor modelo obtenido, el margen de error promedio puede interpretarse a partir de métricas como el MAE o el RMSE. Este valor representa la desviación promedio entre las ventas reales y las predichas, lo que permite estimar el nivel de incertidumbre en las predicciones.

Si el error promedio es relativamente bajo en comparación con la magnitud de las ventas, el modelo puede considerarse adecuado para apoyar la toma de decisiones operativas y tácticas, como la planificación de inventarios o la asignación de recursos.

Sin embargo, si el RMSE es significativamente mayor que el MAE, esto indica la presencia de errores grandes (outliers), lo cual puede ser crítico en contextos donde una mala predicción implique pérdidas económicas relevantes.

En términos de decisiones presupuestales, el modelo puede considerarse **suficientemente robusto** siempre que:

- Mantenga un buen desempeño en datos de prueba (generalización)
- Presente errores controlados y consistentes
- No evidencie sobreajuste

En conclusión, el modelo es útil como herramienta de apoyo para la toma de decisiones, pero debe complementarse con análisis de riesgo, especialmente en escenarios donde los errores de predicción puedan generar impactos financieros significativos.

3.4. Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

En esta fase, el objetivo es comprender la mecánica interna de una Red Neuronal Multicapa. En lugar de predecir un valor continuo de ventas, clasificaremos el éxito de una campaña en tres categorías: **Bajo Impacto** (C_1), **Impacto Medio** (C_2) e **Impacto Alto** (C_3).

Arquitectura y Contexto

Considere una red neuronal con la siguiente estructura:

- **Capa de Entrada:** 3 características (x_1 : TV, x_2 : Radio, x_3 : Newspaper).
- **Capa Oculta (Capa 1):** 3 neuronas con función de activación **Sigmoide** (σ).
- **Capa de Salida (Capa 2):** 3 neuronas (una por clase) con función de activación **Softmax**.

Definición de Parámetros y Datos

Se proporcionan los siguientes pesos y sesgos (*biases*) ya entrenados para un mercado específico. El vector de entrada normalizado es $X = [0,5, 0,3, 0,2]^T$.

Capa 1 (Oculta):

$$W^{(1)} = \begin{pmatrix} 0,2 & 0,5 & -0,1 \\ 0,8 & -0,3 & 0,4 \\ -0,5 & 0,1 & 0,2 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0,1 \\ -0,2 \\ 0,0 \end{pmatrix}$$

Capa 2 (Salida):

$$W^{(2)} = \begin{pmatrix} 0,5 & -0,4 & 0,1 \\ 0,2 & 0,8 & -0,5 \\ -0,1 & 0,3 & 0,7 \end{pmatrix}, \quad b^{(2)} = \begin{pmatrix} -0,1 \\ 0,2 \\ 0,1 \end{pmatrix}$$

Tarea de Cálculo Manual

Realice los cálculos paso a paso para determinar a qué categoría pertenece la campaña publicitaria:

1. **Puntaje de la Capa Oculta ($Z^{(1)}$):** Calcule el producto punto $Z^{(1)} = W^{(1)}X + b^{(1)}$.
2. **Activación de la Capa Oculta ($A^{(1)}$):** Aplique la función sigmoide a cada elemento de $Z^{(1)}$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
3. **Puntaje de la Capa de Salida ($Z^{(2)}$):** Calcule $Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}$.
4. **Probabilidades de Clase ($A^{(2)}$):** Aplique la función **Softmax** al vector resultante $Z^{(2)}$ para obtener las probabilidades de cada clase (P_1, P_2, P_3):

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

FASE 4: Datos.

$$x = (0,5 ; 0,3 ; 0,2)^T$$

① Capa 1: $w^{(1)} = \begin{pmatrix} 0,2 & 0,5 & -0,1 \\ 0,8 & -0,3 & 0,4 \\ -0,5 & 0,1 & 0,2 \end{pmatrix}, b^{(1)} = \begin{pmatrix} 0,1 \\ -0,2 \\ 0,0 \end{pmatrix}$

② Capa 2: $w^{(2)} = \begin{pmatrix} 0,5 & -0,4 & 0,1 \\ 0,2 & 0,8 & -0,5 \\ -0,1 & 0,3 & 0,7 \end{pmatrix}, b^{(2)} = \begin{pmatrix} -0,1 \\ 0,2 \\ 0,1 \end{pmatrix}$

Capa oculta $z(1) = w^{(1)} x + b^{(1)}$.

③ Neurona 1:

$$(0,2)(0,5) + (0,5)(0,3) + (0,1)(0,2) =$$

$$0,1 + 0,15 - 0,02 = 0,23.$$

$$z_1^{(1)} = 0,23 + 0,1 = 0,33.$$

④ Neurona 2:

$$(0,8)(0,5) + (0,3)(0,3) + (0,4)(0,2) =$$

$$(0,4) - (0,09) + (0,08) = 0,39$$

$$z_2^{(1)} = 0,39 - 0,2 = 0,19$$

⑤ Neurona 3:

$$(-0,5)(0,5) + (0,1)(0,3) + (0,2)(0,2) =$$

$$-0,25 + 0,03 + 0,04 = -0,18$$

$$z_3^{(1)} = -0,18 + 0 = -0,18.$$

$$z^{(1)} = (0,33; 0,19; -0,18)^T.$$

⑥. Sigmoide $A^{(1)} =$

$$g(z) = \frac{y}{y + e^{-z}}$$

$$g(0,33) \approx 0,582$$

$$g(0,19) \approx 0,547$$

$$g(-0,18) \approx 0,455.$$

$$A^{(1)} \approx (0,582; 0,547; 0,455)^T$$

Capa de salida $z^{(2)} = w^{(2)} \cdot A^{(1)} + b^{(2)}$

→ Neurona 1:

$$(0,5)(0,582) - (0,4)(0,547) + (0,1)(0,455) =$$

$$0,291 - 0,219 + 0,0455 = 0,1175.$$

$$z_1^{(2)} = 0,1175 - 0,1 = 0,0175.$$

→ Neurona 2:

$$(0,2)(0,582) + (0,8)(0,547) - (0,5)(0,455) =$$

$$0,1164 + 0,4376 - 0,2275 = 0,3265.$$

$$z_2^{(2)} = 0,3265 + 0,2 = 0,5265$$

→ Neurona 3:

$$(-0,1)(0,582) + (0,3)(0,547) + (0,7)(0,455) =$$

$$-0,0582 + 0,1641 + 0,3185 = 0,4244$$

$$z_3^{(2)} = 0,4244 + 0,1 = 0,5244.$$

$$z^{(2)} = (0,0175, 0,5265, 0,5244)^T$$

* Softmax (Probabilidades):

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^J e^{z_j}}$$

$$e^{0,013} \approx 1,018$$

$$e^{0,527} \approx 1,693$$

$$e^{0,524} \approx 1,689$$

$$\text{Suma} = 1,018 + 1,693 + 1,689 = 4,400$$

Probabilidad =

$$P_1 = \frac{1018}{4400} = 0,231$$

$$P_2 = \frac{1693}{4400} = 0,385$$

$$P_3 = \frac{1689}{4400} = 0,384$$

$$(P_1, P_2, P_3) = (0,231), (0,385), (0,384)$$

Pregunta 1:

La mayor probabilidad es P2 0.385 por lo tanto la red se clasifica como impacto medio clase 2.

Pregunta 2:

Se utilizamos softmax en la ultima copa porque convierte los valores en probabilidades reales, todas las salidas son positivas y suman exactamente 1, esto permite interpretar la salida como una distribución de probabilidad. Que pasaría si se usara una función lineal no se limitarían los valores y no se podría interpretar como probabilidades, que pasaría si es sigma de cada salida estaría entre 0 y 1 pero no garantiza que sumen 1.

3.5. Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

En la manufactura, una parada de máquina no planificada es un fallo crítico que genera cuellos de botella y sobrecostos. En esta fase, evaluaremos el desempeño de la Red Neuronal entrenada para el **Mantenimiento Predictivo (PdM)**, donde el objetivo es clasificar si una máquina fallará (Clase 1) o no (Clase 0).

Para el desarrollo de esta sección, utilice el notebook disponible en:
GitHub: Semana 8-9 — Clasificación y Evaluación de Modelos.

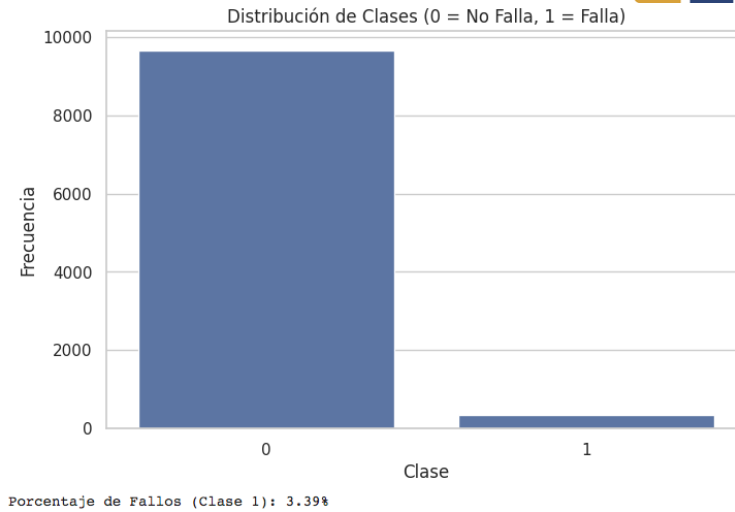
El Desafío del Desbalance de Clases

En plantas reales, las máquinas fallan rara vez (menos del 5 % del tiempo). Esto genera un *data-set* desbalanceado. Observe en el código del notebook cómo se calculó el diccionario `class_weights_dict` para manejar esta situación.

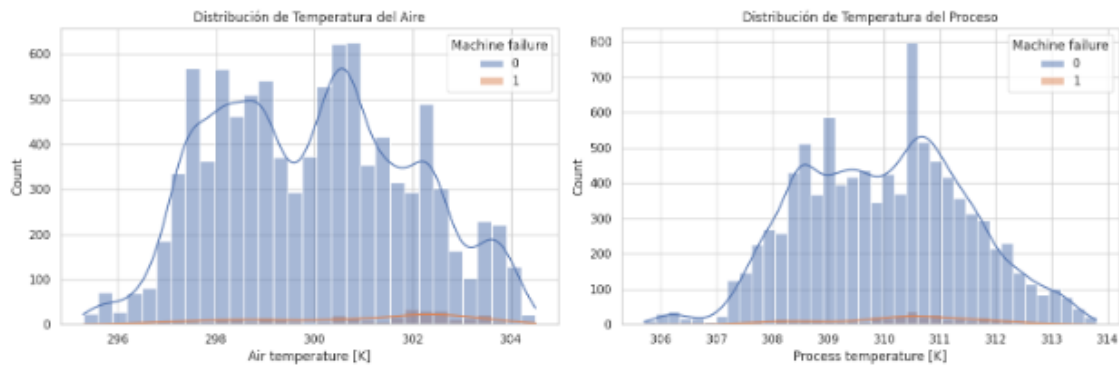
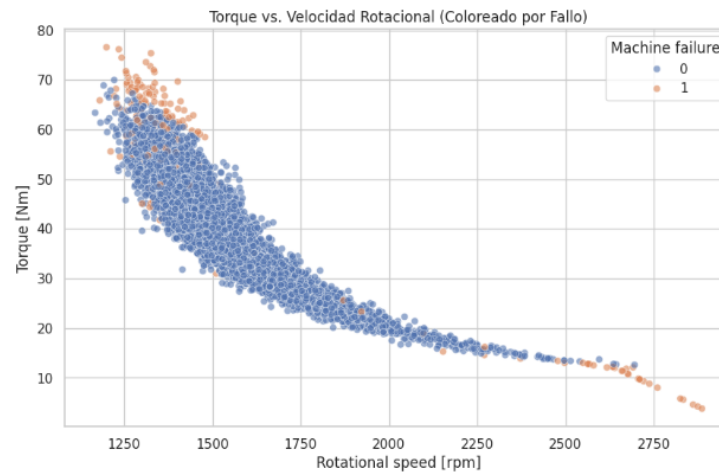
Preguntas Fase 5A:

- **Análisis de Pesos:** Si el modelo indica que los errores en la Clase 1 (Fallo) pesarán aproximadamente 14.7 veces más que en la Clase 0, ¿qué intenta corregir el algoritmo al configurar el parámetro `class_weight` durante el entrenamiento?
- **La Trampa de la Exactitud:** Explique por qué en este caso un *Accuracy* del 95 % podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales.

```
--- Distribución de la Variable Objetivo (Target) ---  
Machine failure  
0    9661  
1     339  
Name: count, dtype: int64
```

Observación: Vemos que solo ~3.39% de los datos corresponden a fallos. ¡Este es un dataset desbalanceado! Esto confirma que el Accuracy no será una buena métrica por sí sola (un modelo que siempre diga "No Falla" tendría 96.6% de accuracy).



Observación: A simple vista, parece que los fallos (Clase 1, en naranja) tienden a ocurrir con valores de Torque más altos y Velocidad Rotacional más baja. Las temperaturas

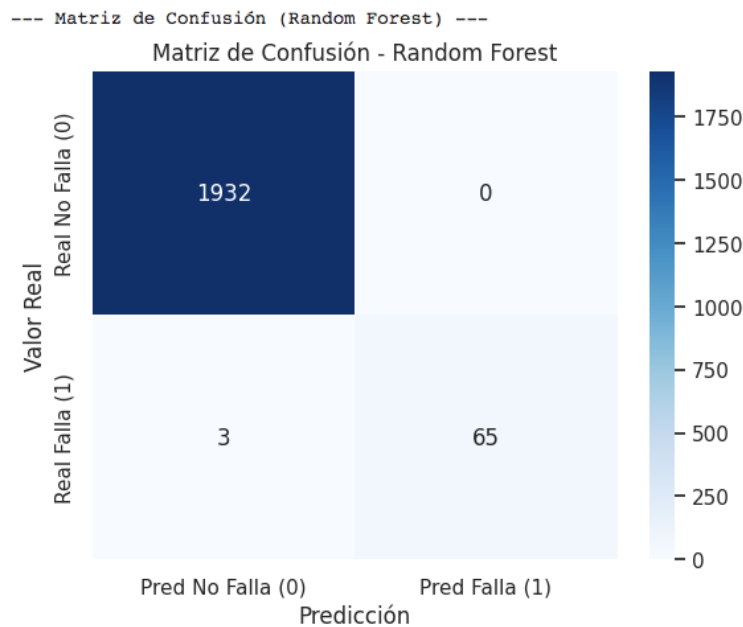
también muestran distribuciones ligeramente diferentes. ¡Buenas noticias! Esto significa que nuestros modelos probablemente puedan encontrar patrones.

--- Evaluación del Modelo: Random Forest ---

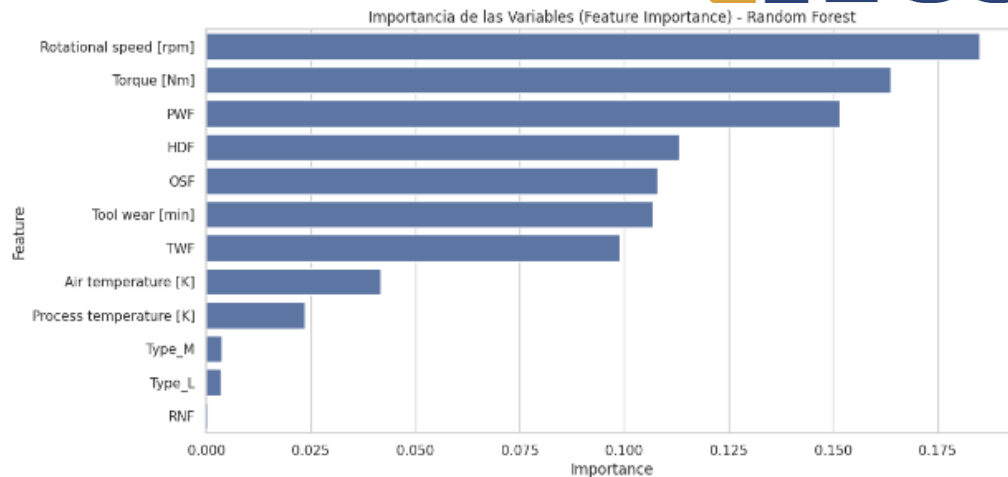
Accuracy (Exactitud): 0.9985

Recall (Sensibilidad) Clase 1: 0.9559

--- Matriz de Confusión (Random Forest) ---



- El Accuracy general es muy alto (ej. ~98-99%).
- Miremos el Recall para la Clase 1 (Fallo): ¡Es alto! (ej. > 0.85 o 85%). Esto significa que de todos los fallos reales que ocurrieron en la prueba, nuestro modelo detectó correctamente más del 85% de ellos.
- La Precisión para la Clase 1 es un poco más baja. Esto significa que a veces genera Falsos Positivos (falsas alarmas), pero como dijimos en el Análisis del Problema, ¡preferimos eso a un Falso Negativo!
- Importancia de las Variables (Features): Una gran ventaja de Random Forest es que nos dice qué variables fueron más importantes para tomar la decisión.



Observación: Es muy probable que Torque, Rotational speed y Tool wear sean las variables más decisivas. Esto tiene mucho sentido en un contexto de ingeniería.

Preguntas de Reflexión y Negocio Fase 5

- **Trade-off entre Recall y Precision:** En el contexto de evitar paradas de línea críticas, ¿preferiría usted un modelo con un *Recall* del 98 % pero una *Precision* del 60 %, o viceversa? Justifique su respuesta basándose en los costos calculados anteriormente.
- **Curva de Aprendizaje:** Observe la gráfica de *loss* (pérdida) y *accuracy* durante el entrenamiento. ¿Considera que 30 épocas son suficientes para que el modelo converja? ¿Qué sucede con el error de validación comparado con el de entrenamiento?
- **Conclusión Final del Taller:** Tras recorrer desde la regresión analítica hasta la clasificación con redes neuronales, ¿cómo puede el uso de métricas avanzadas (como el Recall) transformar la rentabilidad de una empresa de manufactura?
- **Análisis de Pesos:** Si el modelo indica que los errores en la Clase 1 (Fallo) pesaran aproximadamente 14.7 veces mas que en la Clase 0, ¿qué intenta corregir el algoritmo al configurar el parámetro class weight durante el entrenamiento?
Pesos de Clase calculados: {0: np.float64(0.51753137533963), 1: np.float64(14.76014760147)
Esto significa que los errores en la Clase 1 (Fallo) pesarán ~14.7 veces más.
- **La Trampa de la Exactitud:** Explique por que en este caso un Accuracy del 95 % podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales.

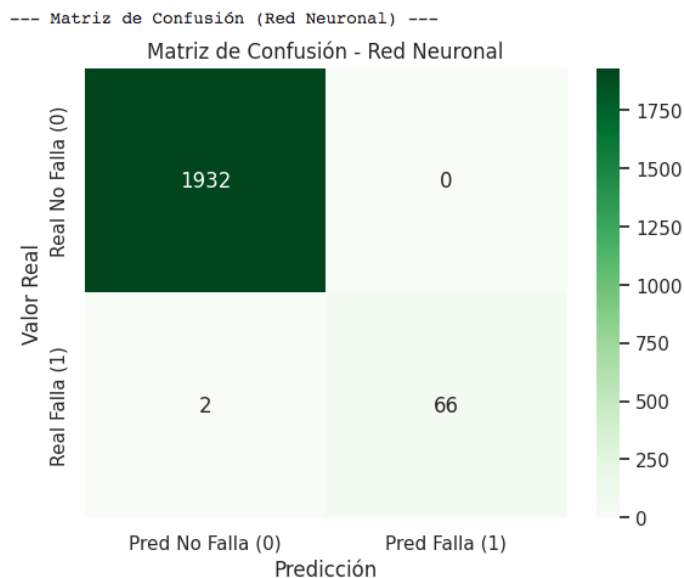
Accuracy (Exactitud): 0.9990

Recall (Sensibilidad) Clase 1: 0.9706

Interpretación de la Matriz de Confusión

Utilice la matriz de confusión generada por el heatmap de Seaborn en su notebook para completar el siguiente análisis. Suponga que para su planta de producción, el costo de una reparación no planificada (Fallo Real no detectado) es de USD 5,000 y el costo de una inspección preventiva innecesaria (Falsa Alarma) es de USD 200.

Tarea de Cálculo de Métricas: A partir de los valores obtenidos en su matriz de confusión(TN, FP, FN, TP), calcule manualmente:



--- Reporte de Clasificación (Red Neuronal) ---

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1932
1	1.00	0.97	0.99	68
accuracy			1.00	2000
macro avg	1.00	0.99	0.99	2000
weighted avg	1.00	1.00	1.00	2000

Análisis Red Neuronal: La red neuronal, gracias al class_weight, también debería alcanzar un Recall alto para la Clase 1, similar o incluso superior al Random Forest (a costa, quizás, de una menor precisión, generando más falsas alarmas).

1. **Precision (Precisión):** $\frac{TP}{TP+FP}$ — ¿Qué tan confiable es el modelo cuando emite una alerta de falla?
2. **Recall (Sensibilidad):** $\frac{TP}{TP+FN}$ — ¿Qué porcentaje de los fallos reales totales logró capturar el modelo?
3. **Costo Total de Error:** Calcule el impacto económico mensual del modelo utilizando la fórmula:

$$\text{Costo} = (FN \times 5000) + (FP \times 200)$$

- Precisión $\frac{TP}{TP+FP} = \frac{66}{66+0} = 1.00$

Cuando el modelo detecta una falla, siempre acierta. Es totalmente confiable en sus alertas.

- Recall (Sensibilidad) $\frac{TP}{TP+FN} = \frac{66}{66+2} = 0.9706$

El modelo logra detectar el 97.06% de las fallas reales, lo cual es bastante alto. Solo se le escapan 2 casos.

- Costo Total de Error: $\text{Costo} = (FN * 5000) + (FP + 200)$
 $\text{Costo} = (2 * 5000) + (0 * 200) = 10.000 \text{ USD}$

Interpretación general

- El modelo es extremadamente preciso (no genera falsas alarmas → ahorro en inspecciones innecesarias).
- Tiene una alta sensibilidad, pero los pocos errores que comete ($FN = 2$) son críticos porque implican:
 - Fallas no detectadas
 - Alto costo operativo

Conclusión

Aunque el modelo es muy bueno, en un entorno productivo puede ser recomendable reducir aún más los falsos negativos, ya que cada uno cuesta USD 5,000, lo que impacta significativamente.

1). En el contexto de evitar paradas de línea críticas, es preferible un modelo con **alto Recall (98%) y menor Precisión (60%)**, en lugar de lo contrario.

Esto se debe a que el **Recall mide la capacidad del modelo para detectar fallos reales**, mientras que la Precisión evalúa qué tan confiables son las alertas generadas. En un entorno industrial, el costo de un **falso negativo (no detectar una falla real)** es significativamente mayor que el de un **falso positivo (falsa alarma)**.

Dado que una falla no detectada puede generar costos elevados (por ejemplo, USD 5,000 por parada no planificada), mientras que una inspección innecesaria tiene un costo mucho menor (USD 200), resulta más conveniente minimizar los falsos negativos, incluso si esto implica un aumento en las falsas alarmas.

En conclusión, priorizar el Recall permite reducir el riesgo de fallos críticos y proteger la continuidad operativa, lo cual tiene un impacto directo en la rentabilidad de la empresa.

2). Para determinar si 30 épocas son suficientes, es necesario analizar el comportamiento conjunto de la pérdida (*loss*) y la precisión (*accuracy*) tanto en entrenamiento como en validación.

Si durante las 30 épocas la pérdida de entrenamiento y validación disminuyen de manera similar y tienden a estabilizarse, se puede concluir que el modelo ha convergido adecuadamente. Sin embargo, si la pérdida de entrenamiento continúa disminuyendo mientras que la de validación se estabiliza o comienza a aumentar, se está presentando un fenómeno de **sobreajuste**.

En este último caso, 30 épocas podrían ser excesivas, ya que el modelo estaría aprendiendo patrones específicos del conjunto de entrenamiento y perdiendo capacidad de generalización.

Por el contrario, si ambas curvas aún muestran tendencia descendente al final de las 30 épocas, esto indicaría que el modelo no ha convergido completamente y podría beneficiarse de más iteraciones.

En conclusión, el análisis de la curva de aprendizaje permite identificar el punto óptimo de entrenamiento, donde se logra el mejor equilibrio entre ajuste y generalización.

3). El uso de métricas avanzadas como el Recall transforma significativamente la rentabilidad de una empresa de manufactura al permitir una evaluación más precisa del desempeño de los modelos en función del impacto real en el negocio.

A diferencia de métricas generales como el Accuracy, que pueden ser engañosas en escenarios con datos desbalanceados, el Recall permite medir la capacidad del modelo para detectar eventos críticos, como fallas en maquinaria. Esto es fundamental en mantenimiento predictivo, donde la omisión de una falla puede generar altos costos operativos, tiempos de inactividad y pérdida de productividad.

Al enfocar el análisis en métricas alineadas con los costos del sistema, las decisiones dejan de basarse únicamente en indicadores estadísticos y pasan a fundamentarse en criterios económicos y operativos.

En este sentido, la integración de técnicas analíticas —desde modelos de regresión hasta redes neuronales— junto con una correcta interpretación de métricas, permite optimizar procesos, reducir riesgos y mejorar la eficiencia global del sistema productivo.

En conclusión, el enfoque basado en métricas avanzadas no solo mejora la precisión técnica de los modelos, sino que se convierte en una herramienta estratégica para incrementar la rentabilidad y sostenibilidad de las operaciones industriales.

